

ZORDON — Modelos de Predição

Análise complementar com evidências reais extraídas do código-fonte do projeto

Disciplina/Trabalho: Identificação de Modelos de Predição no ZORDON | **Sistema analisado:** ZORDON Intelligence Platform (API Node.js/Express + Engine de Decisão) | **Repositório:** apps/api/src/engine, apps/api/src/services

Este documento complementa a análise comparativa inicial entre modelos de predição clássicos e o sistema ZORDON. A diferença deste material é que cada classificação ("Sim", "Não" ou "Parcialmente") foi verificada diretamente no código-fonte real do projeto, e não apenas por inferência teórica. Os trechos de código exibidos abaixo foram extraídos sem alterações dos arquivos do repositório, com o caminho do arquivo indicado acima de cada bloco.

1. Tabela Comparativa Completa

A tabela abaixo reproduz a análise original e preenche as colunas que haviam ficado em branco ("Propriedades do Modelo" e "Quais propriedades tem no Zordon"), agora com base na leitura direta do código.

Modelo	Como Funciona	Propriedades do Modelo	Usa IA/Aprendizado?	Existe no ZORDON?	Como aparece no ZORDON	Quais propriedades tem no ZORDON
Regressão Linear	Cálculo matemático que prevê valores futuros com base em variáveis históricas.	Relação linear entre variável dependente e independentes; coeficientes ajustados por mínimos quadrados; exige normalidade e homocedasticidade dos resíduos.	Não	Não	Não localizado em nenhum arquivo do projeto.	— nenhuma propriedade encontrada.
Séries Temporais (ARIMA, Forecasting)	Analisa o comportamento dos dados ao longo do tempo para prever cenários futuros.	Componentes de tendência, sazonalidade e ruído; modelos autorregressivos (AR) e de médias móveis (MA); exige série estacionária.	Parcial	Parcialmente	<code>trendAnalyzer.js</code> conta repetições de um mesmo código de decisão no histórico; <code>quedaVendas.prediction.js</code> compara as 3 últimas vendas em sequência.	Observação de tendência por contagem de frequência e comparação sequencial simples (<code>q[0]>q[1]>q[2]</code>) — sem AR, MA, sazonalidade calculada ou estacionariedade.
Machine Learning	Algoritmos aprendem padrões automaticamente a partir dos dados.	Aprendizado via dados de treino, generalização para novos casos, otimização de parâmetros (ex.: gradiente descendente).	Sim	Não	Nenhum pipeline de treinamento, dataset de treino ou modelo serializado encontrado no projeto.	— nenhuma propriedade encontrada.
Redes Neurais	Simulam conexões neurais para identificar padrões complexos nos dados.	Camadas de neurônios artificiais, pesos ajustados por backpropagation, funções de ativação não lineares.	Sim	Não	Não há estrutura de camadas, pesos treináveis nem bibliotecas de deep learning no projeto.	— nenhuma propriedade encontrada.

Árvore de Decisão	Usa estruturas condicionais para tomar decisão com base em critérios.	Estrutura hierárquica de nós de decisão, divisão por critério (ex.: entropia/Gini), caminhos condicionais até um nó-folha.	Pode usar	Parcialmente	<code>ruleEvaluator.js</code> , função <code>avaliarBloco()</code> , avalia blocos de condições aninhados com operadores AND/OR.	Ramificação condicional (AND/OR) recursiva, mas com regras fixas — não há indução automática da árvore a partir de dados (sem cálculo de entropia/Gini).
Sistema Especialista Baseado em Regras	Usa regras pré-definidas para gerar inferências e apoiar decisões.	Base de conhecimento explícita (regras SE-ENTÃO) e motor de inferência que aplica essas regras sobre os fatos, sem exemplos de treino.	Não	Sim	<code>engine.js</code> carrega regras dinamicamente; <code>ruleEvaluator.js</code> avalia regras dinâmicas do banco; arquivos em <code>engine/rules/*</code> contêm as regras fixas.	Base de conhecimento dupla: regras dinâmicas em JSONB (tabela <code>regras</code>) + regras fixas em arquivo, todas avaliadas por um motor de inferência central.
Modelo Heurístico	Usa critérios práticos para encontrar soluções rápidas e eficientes.	Critérios simplificados (regras de bolso) e pesos para chegar a uma solução satisfatória, não necessariamente ótima.	Não	Sim	<code>scoreCalculator.js</code> , <code>score.service.js</code> e <code>escalationCalculator.js</code> atribuem pesos a impacto, urgência, recorrência e confiança.	Pontuação por pesos fixos (ex.: $\text{Impacto} \times 0.40 + \text{Urgência} \times 0.25 + \text{Recorrência} \times 0.20 + \text{Confiança} \times 0.15$) e faixas de corte para classificar criticidade.
Sistema de Apoio à Decisão (DSS)	Auxilia o usuário na análise de informações e na tomada de decisão.	Integra dados, modelos analíticos e interface para apoiar — não substituir — a decisão humana.	Não necessariamente	Sim	<code>intelligence.service.js</code> (<code>montarInteligencia</code>) consolida decisões, tendências, previsões e cenários num painel único.	Gera <code>recomendacao_principal</code> , <code>ranking_top</code> e resumo executivo, exibidos na página "Inteligência" do frontend.
Modelo Preditivo Probabilístico	Trabalha com probabilidades matemáticas para prever eventos futuros.	Distribuições de probabilidade, inferência bayesiana ou frequentista, intervalos de confiança.	Pode usar	Não	Nenhum cálculo de probabilidade, distribuição estatística ou intervalo de confiança encontrado.	— nenhuma propriedade encontrada.
Inferência Determinística	Produz resultados previsíveis através de regras fixas e condicionais.	Mesma entrada sempre gera a mesma saída; regras fixas; ausência de aleatoriedade ou incerteza.	Não	Sim	Toda a engine (<code>engine.js</code> + <code>ruleEvaluator.js</code>) segue esse princípio: operadores fixos, sem números aleatórios.	Mapa fechado de operadores (<code>></code> , <code>>=</code> , <code><</code> , <code><=</code> , <code>=</code> , <code>!=</code>) aplicado sobre um contexto calculado deterministicamente a partir dos dados de produto e venda.

2. Evidências no Código-Fonte

Nesta seção estão os trechos reais de código que sustentam as classificações "Sim" e "Parcialmente" da tabela acima, organizados por modelo.

2.1 Sistema Especialista Baseado em Regras

A engine principal carrega dinamicamente todas as regras de arquivos separados por categoria (problemas, alertas, oportunidades e previsões) e executa cada uma contra os dados atuais:

apps/api/src/engine/engine.js

```
const TIPOS = ["problems", "alerts", "opportunities", "predictions"];

async function carregarRules() {
  const rules = [];
  for (const tipo of TIPOS) {
    const pasta = path.join(RULES_PATH, tipo);
    if (!fs.existsSync(pasta)) continue;
    const arquivos = fs.readdirSync(pasta);
    for (const arquivo of arquivos) {
      if (!arquivo.endsWith(".js")) continue;
      const module = await import(`file://${path.join(pasta, arquivo)}`);
      const rule = module.default;
      if (typeof rule === "function") rules.push(rule);
    }
  }
  return rules;
}

export default async function engine(data) {
  const rules = await carregarRules();
  let resultados = [];
  for (const rule of rules) {
    const resultado = await rule(data);
    if (Array.isArray(resultado)) resultados.push(...resultado);
    else if (resultado) resultados.push(resultado);
  }
  resultados = ordenar(resultados);
  return resultados;
}
```

Além das regras fixas em arquivo, o ZORDON também suporta uma base de regras **dinâmica**, guardada no banco de dados em formato JSONB e interpretada em tempo de execução — característica clássica de sistema especialista, em que a base de conhecimento pode ser atualizada sem alterar o código:

apps/api/src/engine/ruleEvaluator.js

```
const OPS = {
  ">": (a, b) => a > b,
  ">=": (a, b) => a >= b,
  "<": (a, b) => a < b,
  "<=": (a, b) => a <= b,
  "=": (a, b) => a === b,
  "!=": (a, b) => a !== b,
};

function avaliarBloco(bloco, ctx) {
  if (!bloco || !Array.isArray(bloco.condicoes)) return false;
  const op = (bloco.operador || "AND").toUpperCase();
```

```

if (op === "OR") {
  return bloco.condicoes.some((c) =>
    c.condicoes ? avaliarBloco(c, ctx) : avaliarCondicao(c, ctx)
  );
}
return bloco.condicoes.every((c) =>
  c.condicoes ? avaliarBloco(c, ctx) : avaliarCondicao(c, ctx)
);
}

function avaliarRegra(regra, produtos, vendas) {
  const resultados = [];
  for (const produto of produtos) {
    const ctx = montarContexto(produto, vendas);
    const bloco = typeof regra.condicoes === "string"
      ? JSON.parse(regra.condicoes) : regra.condicoes;

    if (!avaliarBloco(bloco, ctx)) continue;
    const impacto_valor = calcularImpacto(regra.impacto_formula, ctx);

    resultados.push({
      tipo: regra.tipo, produto_id: produto.id, codigo: regra.codigo,
      titulo: renderizar(regra.titulo_template, ctx, impacto_valor),
      impacto_valor,
      prioridade: regra.prioridade || "MEDIA",
      score: impacto_valor * (Number(regra.peso) || 1),
      origem: "db",
    });
  }
  return resultados;
}

```

Exemplo de regra fixa que representa conhecimento de negócio explícito (SE produto está parado E estoque acima do mínimo, ENTÃO é um problema de capital parado):

apps/api/src/engine/rules/problems/produtoParado.rule.js

```

export default function produtoParadoRule({ produtos = [], vendas = [] }) {
  const resultados = [];
  const hoje = new Date();

  for (const produto of produtos) {
    const vendasProduto = vendas.filter((v) => v.produto_id === produto.id);
    if (vendasProduto.length === 0) continue;

    const ultimaVenda = vendasProduto.sort((a, b) => new Date(b.data) - new Date(a.data))[0];
    const diasSemVenda = (hoje - new Date(ultimaVenda.data)) / (1000 * 60 * 60 * 24);

    const valor = Number(produto.valor || 0);
    const estoque = Number(produto.estoque || 0);
    const minimo = Number(produto.minimo || 0);

    // REGRA REAL
    if (diasSemVenda > 30 && estoque > minimo) {
      const impacto = valor * estoque;
      resultados.push({
        tipo: "problema", codigo: "PRODUTO_PARADO",
        titulo: "Produto parado",
        descricao: `${produto.nome} está há ${Math.floor(diasSemVenda)} dias sem vendas`,
        impacto_valor: impacto || 0,
        recomendacao: { acao: "Criar campanha promocional ou aplicar desconto" },
        prioridade: impacto > 1000 ? "ALTO" : "MEDIO",
        score: 400
      });
    }
  }
}

```

```
}
  return resultados;
}
```

2.2 Modelo Heurístico

O ZORDON não calcula prioridade por métodos estatísticos: ele usa pesos e faixas de corte definidos manualmente — a definição clássica de heurística (regra prática que funciona bem na maioria dos casos, sem garantia formal de otimalidade):

apps/api/src/engine/utils/scoreCalculator.js

```
export function calcularScore(decisao) {
  let score = 0;

  // 1. Impacto financeiro (principal fator)
  score += Number(decisao.impacto_valor || 0);

  // 2. Prioridade
  const prioridadePeso = { CRITICO: 2000, ALTO: 1000, MEDIO: 500, BAIXO: 100 };
  score += prioridadePeso[decisao.prioridade] || 0;

  // 3. Tipo de decisão
  if (decisao.tipo === "problema") score += 500;

  // 4. Recorrência (quanto mais recorrente, mais crítico)
  if (decisao.recorrencia) score += decisao.recorrencia * 200;

  return score;
}
```

apps/api/src/services/score.service.js

```
// Score = (Impacto × 0.40) + (Urgência × 0.25) + (Recorrência × 0.20) + (Confiança × 0.15)
const PESOS = { impacto: 0.40, urgencia: 0.25, recurrence: 0.20, confianca: 0.15 };
const URGENCIA_MAPA = { CRITICA: 100, ALTA: 75, MEDIA: 50, BAIXA: 25 };

function scoredecisao(d, maxImpacto, maxRecorrencia) {
  const dimImpacto = normalizar(Number(d.impacto_valor || 0), maxImpacto);
  const dimUrgencia = URGENCIA_MAPA[d.prioridade] ?? 50;
  const dimRecorrencia = normalizar(Number(d.recorrencia || 0), maxRecorrencia);
  const dimConfianca = Number(d.performance ?? 0.7) * 100;

  const score = Math.round(
    dimImpacto * PESOS.impacto + dimUrgencia * PESOS.urgencia +
    dimRecorrencia * PESOS.recorrencia + dimConfianca * PESOS.confianca
  );
  return { score: Math.min(100, score), breakdown: { ... } };
}

function nivelSaude(score) {
  if (score >= 80) return { nivel: "CRÍTICO", cor: "#F87171" };
  if (score >= 60) return { nivel: "ALTO", cor: "#FBBF24" };
  if (score >= 40) return { nivel: "MODERADO", cor: "#818CF8" };
  if (score >= 20) return { nivel: "BAIXO", cor: "#34D399" };
  return { nivel: "SAUDÁVEL", cor: "#34D399" };
}
```

apps/api/src/engine/utils/escalationCalculator.js

```
export function calcularEscalonamento(decision) {
  const { impacto_valor = 0, recurrence = 0 } = decision;
  let nivel = "BAIXO";
  let acao_recomendada = "Monitorar situação";
```

```
// REGRA PRINCIPAL: impacto financeiro
if (impacto_valor >= 1000) { nivel = "ALTO"; acao_recomendada = "Ação imediata necessária"; }
else if (impacto_valor >= 300) { nivel = "MEDIO"; acao_recomendada = "Planejar ação"; }

// REGRA DE RECORRÊNCIA (override)
if (recorrencia >= 3) { nivel = "CRITICO"; acao_recomendada = "Intervenção urgente"; }

return { nivel, acao_recomendada };
}
```

2.3 Inferência Determinística

O conjunto de operadores do `RuleEvaluator` (seção 2.1) já demonstra a inferência determinística: para a mesma entrada (mesmo produto, mesmas vendas), o operador sempre retorna o mesmo booleano e, portanto, a mesma decisão é sempre gerada — não há números aleatórios, amostragem ou variabilidade entre execuções.

Evidência direta: a função `avaliarCondicao()` em `ruleEvaluator.js` apenas resolve dois valores numéricos do contexto e aplica uma função pura do mapa `OPS` — sem qualquer fonte de aleatoriedade (`Math.random` não é usado em nenhuma regra).

2.4 Árvore de Decisão (parcial)

A recursão de `avaliarBloco()` (mostrada na seção 2.1) percorre uma estrutura de condições aninhadas em AND/OR — funcionalmente equivalente a percorrer os nós de uma árvore de decisão até chegar a um resultado (folha). A diferença em relação ao modelo de Machine Learning "Árvore de Decisão" é que, no ZORDON, essa árvore é **escrita manualmente** (via JSON de condições) e **não induzida automaticamente** a partir de dados de treino (sem cálculo de entropia ou índice Gini).

2.5 Séries Temporais / Comportamento Preditivo Emergente

O ZORDON não implementa ARIMA nem qualquer modelo estatístico de séries temporais formal, mas possui quatro mecanismos que observam o histórico e **projetam** um cenário futuro — o que o documento original chama de "comportamento preditivo emergente":

`apps/api/src/engine/analytics/trendAnalyzer.js`

```
export function analisarTendencias(historico = []) {
  const tendencias = [];
  const mapa = {};

  for (const h of historico) {
    mapa[h.codigo] = (mapa[h.codigo] || 0) + 1;
  }

  for (const codigo in mapa) {
    if (mapa[codigo] >= 3) {
      tendencias.push({
        tipo: "tendencia",
        titulo: "Padrão recorrente identificado",
        descricao: `${codigo} aparece com frequência`,
        intensidade: mapa[codigo],
        impacto: mapa[codigo] * 100
      });
    }
  }
  return tendencias;
}
```

`apps/api/src/engine/rules/predictions/quedaVendas.prediction.js`

```
export default function quedaVendasPrediction({ produtos = [], vendas = [] }) {
  const resultados = [];

  for (const produto of produtos) {
```

```

const vendasProduto = vendas
  .filter((v) => v.produto_id === produto.id)
  .sort((a, b) => new Date(a.data) - new Date(b.data));

if (vendasProduto.length < 3) continue;

const ultimas = vendasProduto.slice(-3);
const q = ultimas.map(v => Number(v.quantidade || 0));

// tendência de queda: 3 últimas vendas em sequência decrescente
const queda = q[0] > q[1] && q[1] > q[2];
if (!queda) continue;

resultados.push({
  tipo: "previsao",
  codigo: "QUEDA_VENDAS_PREVISTA",
  titulo: "Queda de vendas prevista",
  descricao: `${produto.nome} apresenta queda nas últimas vendas`,
  impacto_valor: Number(produto.valor || 0) * Number(produto.estoque || 0),
  recomendacao: { acao: "Revisar preço ou iniciar campanha" },
  prioridade: "MEDIO",
  score: 350
});
}
return resultados;
}

```

apps/api/src/engine/analytics/predictor.js

```

export function preverProblemas(decisions = []) {
  const previsoes = [];

  for (const d of decisions) {
    // QUALQUER PROBLEMA COM IMPACTO ALTO
    if (d.tipo === "problema" && Number(d.impacto_valor) > 500) {
      previsoes.push({
        tipo: "risco_financeiro",
        titulo: "Impacto financeiro relevante detectado",
        descricao: `${d.titulo} pode gerar perdas contínuas`,
        impacto_estimado: d.impacto_valor
      });
    }
    // SCORE ALTO
    if (Number(d.score_final) > 800) {
      previsoes.push({
        tipo: "prioridade_critica",
        titulo: "Ação urgente necessária",
        descricao: `${d.titulo} exige atenção imediata`,
        impacto_estimado: d.impacto_valor
      });
    }
  }
  return previsoes;
}

```

apps/api/src/engine/analytics/scenarioSimulator.js

```

export function simularCenarios(decisions = []) {
  const cenarios = [];

  for (const d of decisions) {
    // PRODUTO PARADO → simular desconto
    if (d.codigo === "PRODUTO_PARADO") {
      cenarios.push({
        codigo: d.codigo, tipo: "simulacao_desconto", atual: d.impacto_valor,

```

```

    simulacoes: [simularDesconto(d, 0.05), simularDesconto(d, 0.1)]
  });
}
// ALTA DEMANDA → simular aumento de preço
if (d.codigo === "PRODUTO_ALTA_DEMANDA") {
  cenarios.push({
    codigo: d.codigo, tipo: "simulacao_preco", atual: d.impacto_valor,
    simulacoes: [simularAumentoPreco(d, 0.05), simularAumentoPreco(d, 0.1)]
  });
}
}
return cenarios;
}

function simularDesconto(decisao, percentual) {
  const impactoAtual = Number(decisao.impacto_valor || 0);
  const ganhoEsperado = impactoAtual * (1 - percentual);
  return { acao: "reduzir_preco", impacto_estimado: ganhoEsperado,
    descricao: `Aplicar desconto de ${percentual * 100}% pode acelerar vendas` };
}

```

Note que `analisarTendencias()` é uma contagem de frequência (não um cálculo estatístico de tendência), `quedaVendasPrediction()` é uma comparação sequencial de 3 pontos (não uma regressão), e `simularCenarios()` é uma simulação "e se" com percentuais fixos (não uma otimização). Por isso a classificação correta é **comportamento preditivo emergente**, e não um modelo formal de séries temporais.

2.6 Sistema de Apoio à Decisão (DSS)

Toda a saída dos módulos anteriores é consolidada em um único painel de inteligência, que organiza, prioriza e recomenda — sem decidir automaticamente no lugar do usuário (as ações de "Resolver"/"Ignorar" continuam manuais, feitas pelo usuário na interface):

`apps/api/src/services/intelligence.service.js`

```

import { analisarTendencias } from "../engine/analytics/trendAnalyzer.js";
import { preverProblemas } from "../engine/analytics/predictor.js";
import { simularCenarios } from "../engine/analytics/scenarioSimulator.js";

export function montarInteligencia({ decisions = [], historico = [] }) {
  const problemas = decisions.filter(d => d.tipo === "problema");
  const oportunidades = decisions.filter(d => d.tipo === "oportunidade");
  const alertas = decisions.filter(d => d.tipo === "alerta");

  const ordenado = [...decisions].sort(
    (a, b) => Number(b.score_final || 0) - Number(a.score_final || 0)
  );

  return {
    resumo: {
      impacto_total: somarImpacto(decisions),
      problemas: problemas.length,
      oportunidades: oportunidades.length,
      alertas: alertas.length
    },
    decisoes: { problemas, oportunidades, alertas },
    insights: {
      tendencias: analisarTendencias(historico),
      previsoes: preverProblemas(decisions),
      cenarios: simularCenarios(decisions)
    },
    top: ordenado.slice(0, 5),
    recomendacao_principal: ordenado[0] || null
  };
}

```


3. Conclusão

O modelo mais próximo do ZORDON é o Sistema Especialista Baseado em Regras. O sistema mantém uma base de conhecimento explícita — regras fixas em arquivo (`engine/rules/*`) e regras dinâmicas em banco de dados, avaliadas por um motor de inferência central (`ruleEvaluator.js` + `engine.js`) — que identifica estoque crítico, produtos sem vendas, risco financeiro e oportunidades comerciais. Não há aprendizado automático: a inferência ocorre de forma determinística.

O Modelo Heurístico também está presente: o ZORDON simplifica decisões usando critérios práticos (pesos fixos) ao classificar criticidade (`escalationCalculator.js`), calcular impacto financeiro e ordenar decisões por relevância (`scoreCalculator.js`, `score.service.js`). A priorização implementada na engine representa exatamente esse mecanismo heurístico de apoio à decisão.

Toda a lógica do ZORDON é baseada em Inferência Determinística: os resultados são previsíveis, a lógica é fixa, e as mesmas regras sempre produzem o mesmo comportamento para a mesma entrada. Exemplo prático verificado no código: quando o estoque fica abaixo do mínimo, a regra de `estoqueBaixo.rule.js` sempre aciona o mesmo alerta; quando um produto fica sem vendas por mais de 30 dias, `produtoParado.rule.js` sempre aciona o alerta de capital parado. Não existe adaptação automática ou aprendizado do sistema.

Embora o ZORDON não utilize modelos formais de predição (sem Regressão Linear, ARIMA, Machine Learning, Redes Neurais ou modelos probabilísticos), o sistema apresenta um **comportamento preditivo emergente**, confirmado em código nos módulos `predictor.js`, `quedaVendas.prediction.js` e `scenarioSimulator.js`: ele analisa padrões operacionais atuais, identifica tendências simples por recorrência, projeta consequências futuras (ex.: risco de ruptura de estoque, queda prevista de vendas) e sugere ações preventivas e cenários alternativos — tudo isso sem aplicar técnicas tradicionais de Inteligência Artificial.

4. Índice de Arquivos Citados

- `apps/api/src/engine/engine.js` — orquestrador da engine, carrega e executa as regras
- `apps/api/src/engine/ruleEvaluator.js` — motor de inferência das regras dinâmicas (DB)
- `apps/api/src/engine/rules/problems/produtoParado.rule.js` — regra fixa de capital parado
- `apps/api/src/engine/rules/problems/produtoSemVendas.rule.js` — regra fixa de produto sem giro
- `apps/api/src/engine/rules/alerts/estoqueBaixo.rule.js` — regra fixa de estoque crítico
- `apps/api/src/engine/rules/opportunities/produtoAltoVenda.rule.js` — regra fixa de oportunidade
- `apps/api/src/engine/rules/predictions/quedaVendas.prediction.js` — previsão de queda de vendas
- `apps/api/src/engine/analytics/trendAnalyzer.js` — análise de tendência por recorrência
- `apps/api/src/engine/analytics/predictor.js` — projeção de risco a partir das decisões atuais
- `apps/api/src/engine/analytics/scenarioSimulator.js` — simulação de cenários "e se"
- `apps/api/src/engine/utils/scoreCalculator.js` — pontuação heurística da decisão
- `apps/api/src/engine/utils/escalationCalculator.js` — nível de escalonamento heurístico
- `apps/api/src/services/score.service.js` — score operacional 0-100 (heurística ponderada)
- `apps/api/src/services/intelligence.service.js` — painel de apoio à decisão (DSS)